Efficient Implementation and Comparative Analysis of Graph Centrality Measures

Project 19

April 25, 2026

Abstract

This report presents a complete implementation pipeline for centrality analysis on unweighted undirected graphs. The system includes exact closeness centrality, exact betweenness centrality using Brandes algorithm, and eigenvalue centrality via power iteration. The implementation is validated through structural tests and optional micro-comparisons against NetworkX on tiny graphs. A synthetic benchmarking framework captures runtime behavior across multiple graph families. Comparative ranking analysis quantifies agreement between centrality measures using top-k overlap, Jaccard similarity, and Spearman rank correlation. A Streamlit dashboard enables interactive graph selection, metric computation, visualization, and score export.

1 Introduction and Problem Statement

Centrality measures quantify node importance from complementary structural perspectives. In practical graph analytics, users need not only algorithm implementations, but also reproducible validation, runtime characterization, and interpretable comparison between metrics. This project addresses these requirements by building a modular, submission-ready analysis stack for three foundational centrality measures.

2 Graph Model and Representation

The project uses unweighted undirected graphs with adjacency-list storage. Nodes are internally represented using contiguous integer IDs for algorithmic efficiency, while external labels are preserved through bidirectional mapping for input/output fidelity.

3 Centrality Definitions

3.1 Closeness Centrality

For node v, with reachable set $R(v)$ and $r = |R(v)|$:

$$S(v) = \sum_{u \in R(v)} d(v, u), \qquad C_{\text{base}}(v) = \frac{r}{S(v)}.$$

The implementation applies reachable-fraction normalization:

$$C(v) = C_{\text{base}}(v) \cdot \frac{r}{n-1}.$$

3.2 Betweenness Centrality

Betweenness centrality of node v is:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}},$$

where σ_{st} is the number of shortest paths between s, t and $\sigma_{st}(v)$ counts those passing through v.

3.3 Eigenvalue Centrality

Eigenvalue centrality satisfies:

$$Ax = \lambda x,$$

where A is the adjacency matrix. Nodes connected to high-score nodes receive higher scores recursively.

4 Algorithms

4.1 BFS-based Closeness

The algorithm runs BFS from each source node to obtain shortest-path distances and computes normalized closeness from the resulting distance sums.

4.2 Brandes Algorithm for Betweenness

For each source node s:

1. BFS discovers shortest-path levels.
2. Track predecessor sets and shortest-path counts σ.
3. Traverse discovered nodes in reverse order to accumulate dependencies.
4. Add source contribution to global betweenness scores.

Undirected double counting is corrected and optional normalization is applied.

4.3 Power Iteration for Eigenvalue

The implementation iterates:

$$x^{(k+1)} = \frac{(A + \alpha I)x^{(k)}}{\|(A + \alpha I)x^{(k)}\|_2},$$

with convergence criterion:

$$\|x^{(k+1)} - x^{(k)}\|_2 < \varepsilon.$$

A small stability shift α improves convergence behavior on bipartite structures.

5 Complexity Analysis

Let $n = |V|$ and $m = |E|$.

Metric	Core Procedure	Time Complexity
Closeness	BFS from every node	$O(n(n+m))$
Betweenness (Brandes)	BFS + dependency accumulation per source	$O(nm)$ (unweighted)
Eigenvalue	Iterative sparse mat-vec	$O(Tm)$ for T iterations

Table 1: Asymptotic complexity of implemented algorithms.

6 Validation Methodology

Validation combines deterministic structure-aware checks and optional reference checks:

- Path graphs: central nodes dominate closeness and betweenness.
- Star graphs: center dominates all three metrics.
- Cycle graphs: symmetry induces equal scores where expected.
- Bridge-community graphs: bridge nodes dominate betweenness.
- Disconnected graphs: methods return safe outputs without crashes.
- Optional NetworkX spot-checks on tiny graphs.

7 Experimental Setup

Benchmarks are run on synthetic graph families:

- path, cycle, star,
- grid,
- Erdős–Rényi,
- Barabási–Albert,
- bridge-community.

Runtime is measured using `time.perf_counter`. Results are exported to CSV and JSON.

8 Benchmark Results

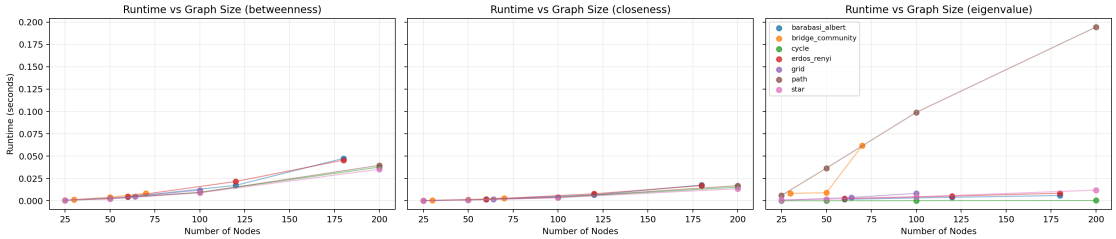

Figure 1: Runtime scaling with graph size for each metric.

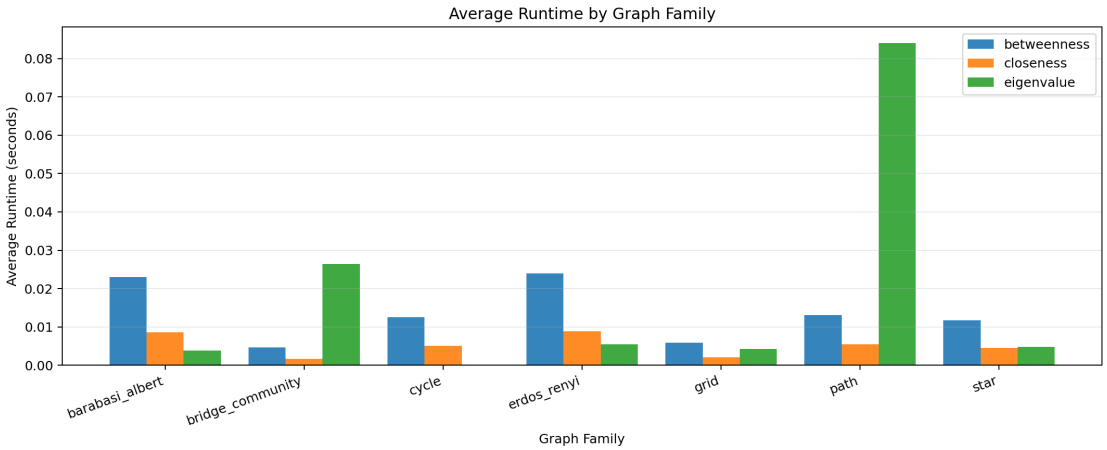

Figure 2: Average runtime comparison across graph families.

The runtime plots show expected scaling trends: closeness and betweenness increase more sharply with graph size due to repeated BFS traversals, while eigenvalue runtime depends primarily on iteration count and graph sparsity.

9 Comparative Analysis

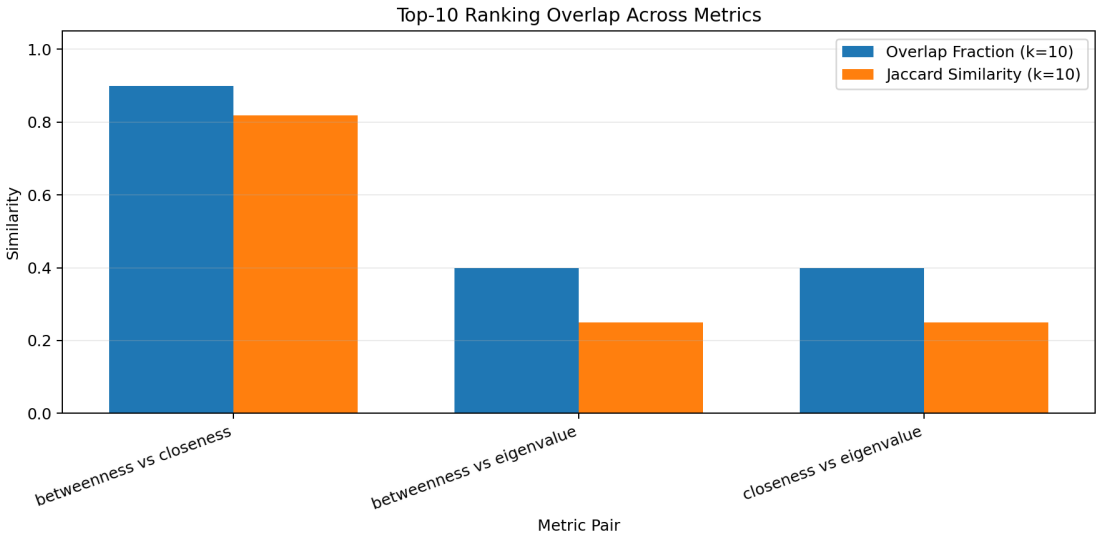

Figure 3: Top-k overlap and Jaccard similarity across metric pairs.

10 Dashboard Description

The Streamlit dashboard supports graph construction and upload, metric selection, summary inspection, graph rendering with centrality-driven encoding, ranking comparison, and CSV export of computed node scores.

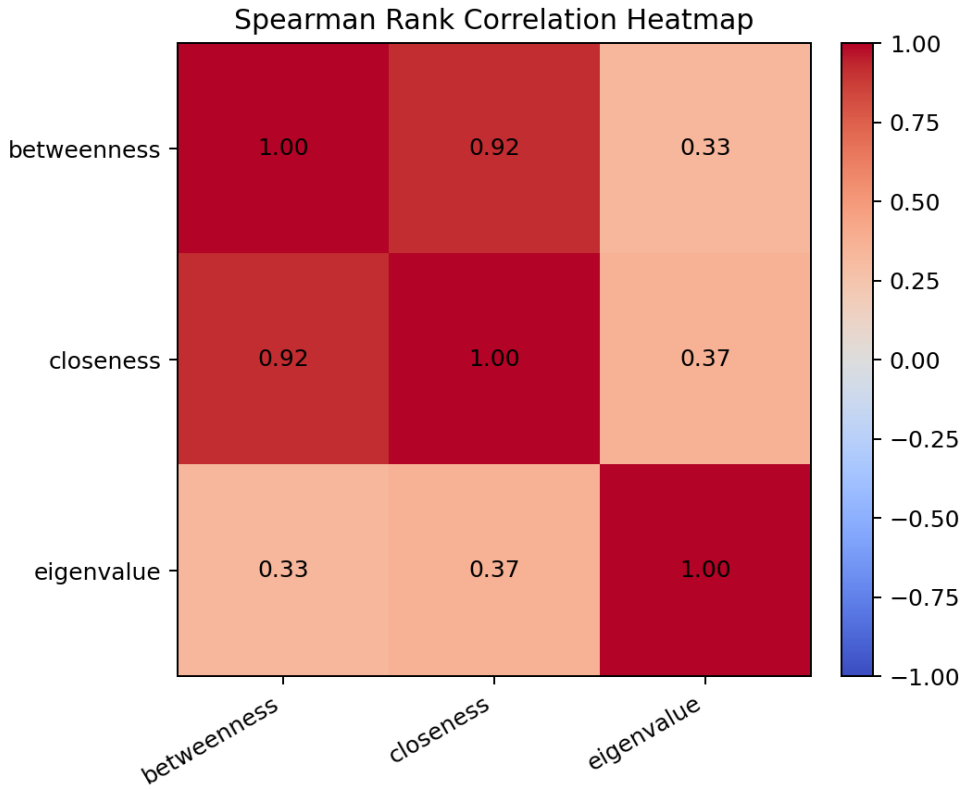

Figure 4: Spearman rank correlation heatmap across centrality rankings.

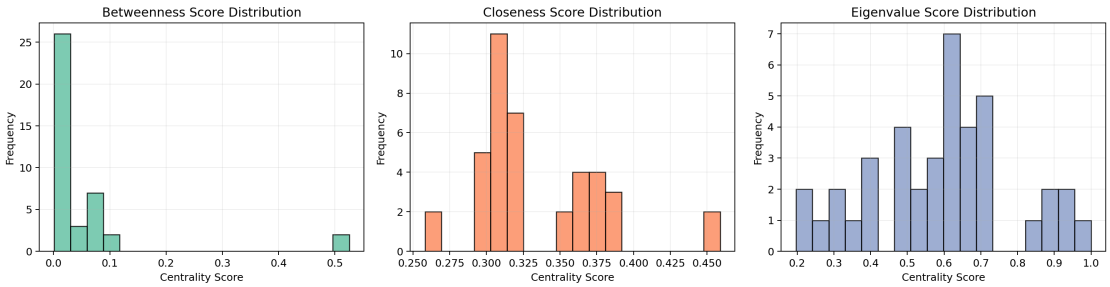

Figure 5: Score distributions for implemented centrality metrics.

11 Discussion

The implemented metrics capture complementary notions of importance: closeness emphasizes global distance efficiency, betweenness captures brokerage on shortest paths, and eigenvalue rewards influence propagated through high-score neighborhoods. Comparative overlap and correlation reveal where these notions align and diverge on different graph structures.

12 Limitations

- Primary scope is unweighted undirected graphs.

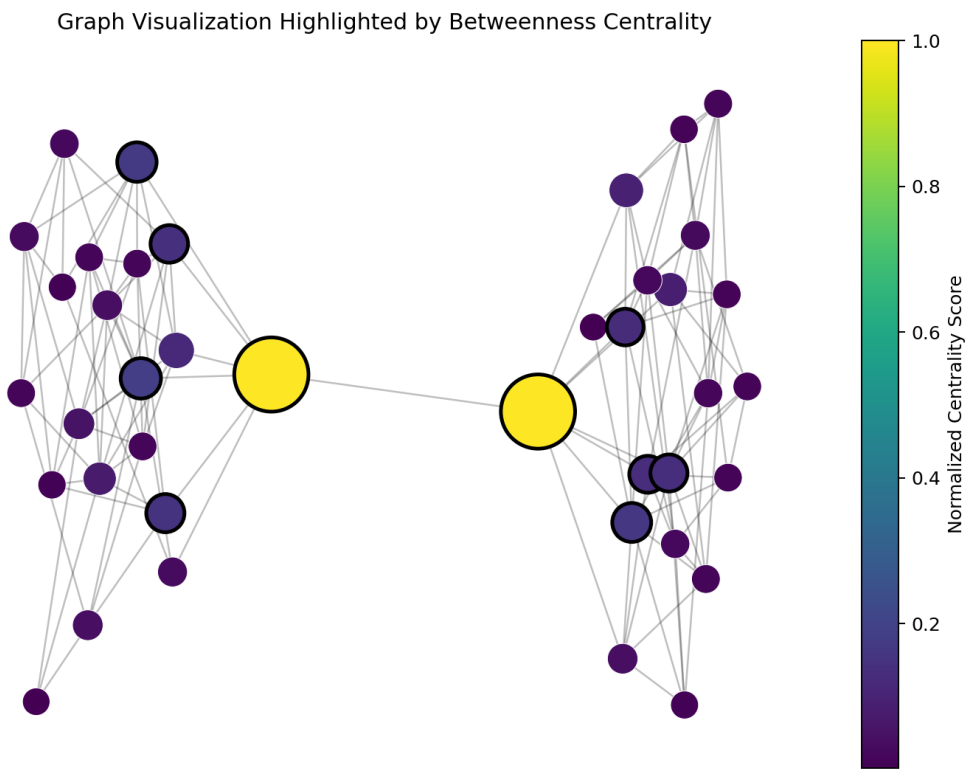

Figure 6: Graph visualization highlighting high-betweenness nodes.

- Benchmarking currently uses synthetic datasets only.
- Interactive dashboard is intended for small-to-medium graphs.
- Advanced statistical comparison methods beyond overlap and correlation are not included.

13 Future Work

- weighted and directed centrality variants,
- larger real-world datasets and ingestion pipelines,
- richer comparative statistics and stability analysis,
- enhanced interactive analytics and reporting features.

14 Conclusion

The project delivers a complete, modular centrality analysis platform with validated algorithm implementations, reproducible benchmarking, ranking comparison tooling, static visualization outputs, and an interactive dashboard. The resulting codebase provides a robust baseline for further comparative network analysis and applied graph analytics.